

Resumen teórico — Sistemas Operativos

Notas del curso: *Sistemas de archivos*

Tomás Rodeghiero

Sistemas Operativos — Universidad Nacional de Río Cuarto

2026

Resumen

Este documento es un resumen teórico del capítulo *Sistemas de archivos* de las *Notas del curso* de Sistemas Operativos (Marcelo Arroyo, UNRC). Se presentan la estructura lógica de un disco, la interfaz de un sistema de archivos (atributos, directorios, operaciones, llamadas al sistema UNIX), el diseño del *Virtual File System* (VFS) y el *common file model*, la representación de archivos y directorios mediante FAT y mediante inodes (incluyendo inodes indirectos), el mecanismo de montaje, la abstracción de dispositivos como archivos especiales en `/dev`, el descriptor de archivo, las operaciones sobre dispositivos de caracteres, el caso de estudio del filesystem por capas de xv6 (con su mecanismo transaccional de logging), el proceso de booting (BIOS, GRUB, UEFI, secure boot, GPT) y los sistemas de archivos especiales (procfs, sysfs, ramfs, initramfs, busybox). El objetivo es servir de material de estudio para preparar los prácticos asociados al sistema de archivos y el final de la materia.

Índice

1. Introducción y estructura lógica de un disco	2
1.1. Particiones y volúmenes	2
1.2. Formato a bajo nivel	3
2. Interfaz de un sistema de archivos	3
2.1. Atributos típicos	3
2.2. Árbol de directorios y paths	3
2.3. Tipos de archivo	4
3. Operaciones sobre archivos y directorios	4
3.1. Operaciones sobre archivos regulares	4
3.2. Operaciones sobre directorios	4
3.3. Paths absolutos y relativos	4
3.4. Llamadas al sistema (UNIX)	5
4. Virtual File System (VFS)	5
4.1. Diseño del VFS	6
4.2. Estructuras de datos del common file model	6
4.3. Servicios comunes provistos por el VFS	6
5. Representación de archivos y directorios	7
5.1. FAT	7
5.2. Nodos índices (inodes)	8
6. Montaje de sistemas de archivos	8
6.1. Ejemplo: montar un pendrive FAT en Linux	9
6.2. Tabla de montaje	9
6.3. Interfaz de FS que el VFS exige	9

6.4. Anatomía de un <code>open</code>	9
6.5. Anatomía de un <code>write</code>	10
7. Dispositivos como archivos especiales	10
7.1. Tabla de device drivers registrados	10
8. Descriptor de archivo	11
9. Operaciones con dispositivos de caracteres	11
9.1. Ejemplo: escribir a <code>stdout</code>	11
9.2. Caso <code>xv6</code>	11
10. Filesystem de <code>xv6</code>	12
10.1. Implementación por capas	12
10.2. FS transaccional con logging	12
10.3. Buffer cache simplificada	12
11. Booting	12
11.1. BIOS	13
11.2. Multiboot loaders (GRUB)	13
11.3. UEFI	13
12. Sistemas de archivos especiales	13
12.1. <code>procfs</code>	14
12.2. <code>sysfs</code>	14
12.3. <code>ramfs</code> e <code>initramfs</code>	14
13. Cierre	14

1. Introducción y estructura lógica de un disco

La memoria principal o RAM es **volátil**: su contenido se pierde al apagar el sistema. Además, tiene capacidad limitada. Por eso se necesita un medio de almacenamiento *masivo y persistente* para los datos y programas del sistema y de los usuarios.

Los dispositivos de almacenamiento han evolucionado a través del tiempo (tarjetas/cintas perforadas, discos magnéticos y ópticos, hasta las actuales memorias no volátiles de estado sólido: pendrives, SSD con interfaz ATA, etc.). En estas notas se denomina genéricamente **disco** a un dispositivo de almacenamiento.

Idea intuitiva

Estos dispositivos almacenan datos en *bloques* de tamaño fijo (por ejemplo, 512 bytes). El sistema operativo no lee ni escribe “bytes sueltos” contra un disco: trabaja siempre con bloques.

1.1. Particiones y volúmenes

Un dispositivo de almacenamiento puede dividirse lógicamente en **particiones** o **volúmenes**. Esto permite implementar sistemas de archivos que no deban manejar un espacio enorme de números de bloques. Un dispositivo no particionado provee *al menos un* volumen.

1.2. Formato a bajo nivel

Un disco se formatea a bajo nivel con la siguiente estructura lógica:

bs	partition table	volume 1	volume 2	...
----	-----------------	----------	----------	-----

- **Boot sector (bs).** Contiene un pequeño programa encargado de cargar el kernel del SO en memoria y transferirle el control.
- **Tabla de partición.** Información (comienzo, fin, booteable, tipo de FS, ...) sobre la división lógica del disco. Cada parte se denomina *partición* o *volumen*.
- **Volúmenes.** En cada uno se almacena un sistema de archivos (FS).

Importante

El estándar BIOS incluye la tabla de particiones primaria (de *cuatro entradas*) dentro del boot sector. El estándar moderno UEFI se basa en una estructura mucho menos restrictiva, descripta más adelante.

Una plataforma de hardware puede incluir varios dispositivos de almacenamiento de distintas tecnologías. El subsistema del SO que provee abstracciones para gestionar objetos de datos y programas se conoce como **sistema de archivos (filesystem)**.

2. Interfaz de un sistema de archivos

El sistema de archivos abstrae una unidad de almacenamiento en un **archivo**. Para organizar la enorme cantidad de archivos de un sistema, éstos se almacenan en contenedores llamados **directorios** o carpetas.

Definición

Un **archivo** se representa como un par (*attributes*, *data*): una colección de *metadatos* (atributos) más una secuencia lógicamente contigua de *datos*, aunque físicamente esos datos pueden estar dispersos en distintos bloques del dispositivo.

2.1. Atributos típicos

- **Nombre.**
- **Tipo** (según el formato lógico del contenido).
- **Tamaño** (comúnmente en bytes).
- **Dueño:** usuario que define el control de acceso.
- **Creador:** usuario que lo creó originalmente.
- **Permisos de acceso:** qué usuarios pueden acceder y cómo (lectura, escritura, ejecución, etc.).
- **Hora y fecha de último acceso;** otros tiempos (creación, modificación).

2.2. Árbol de directorios y paths

Los directorios contienen archivos y *subdirectorios*, formando una estructura de **árbol**: los archivos son las hojas y los directorios los nodos intermedios. Todo sistema de archivos contiene al menos un **directorio raíz**:

- En SO tipo UNIX se denota `/`.
- En MS-Windows se denota `\` (*backslash*).

Para identificar unívocamente a un archivo se usa su **path** (camino desde la raíz), por ejemplo `/home/os-student/books/S0-book.pdf`. Esto permite reusar nombres en distintas carpetas.

2.3. Tipos de archivo

En DOS o MS-Windows, el tipo del archivo se determina por su *extensión* (sufijo del nombre): `.com`, `.exe`, etc. En los SO tipo UNIX, el tipo está en los *metadatos* y la extensión es sólo una convención entre usuarios; el sistema no la interpreta (por ejemplo, `ld-linux.so.2`). UNIX clasifica los archivos en:

- **Regulares:** archivos de datos y ejecutables.
- **Directorios:** contenedores de archivos y subdirectorios.
- **Enlaces simbólicos:** referencias (alias) a otros archivos.
- **Pipes y FIFOs:** usados para IPC.
- **Especiales:** representan dispositivos (de bloques o caracteres).
- **Sockets:** mecanismo de comunicación local o remoto.

3. Operaciones sobre archivos y directorios

3.1. Operaciones sobre archivos regulares

- Creación, lectura, escritura, ejecución, borrado (*delete*).
- **Append:** escribir datos al final.
- **Truncado:** borrar datos al final.
- **Posicionado:** mover el cursor de lectura/escritura.
- **Renombrado:** cambiar el nombre.
- **Locking:** obtener acceso exclusivo.
- Cambiar otros metadatos (dueño, permisos, ...).

3.2. Operaciones sobre directorios

- **Creación.** Es común que un directorio vacío se cree con *dos enlaces*: `..` (referencia al directorio padre) y `.` (referencia al directorio actual).
- **Búsqueda** de un archivo o subdirectorio. Cada elemento dentro de un directorio se llama *directory entry*.
- **Eliminación.** Comúnmente se exige que el directorio esté vacío.

3.3. Paths absolutos y relativos

En las operaciones que reciben un **path**, éste puede ser un **full path** (desde la raíz) o **relativo** al directorio corriente. Por ejemplo:

- `open("/home/user1/docs/mycv.tex")` usa un full path.
- `open("docs/mycv.tex")` refiere al mismo archivo si el directorio corriente es `/home/user1/`.

Variables de ambiente. Un programa se ejecuta en un *ambiente* representado por un conjunto de **variables de ambiente**, cada una un par **variable=value**. En UNIX, un programa accede a ellas mediante el tercer argumento de **main**. El directorio corriente está representado por la variable `PWD` (`echo $PWD`).

3.4. Llamadas al sistema (UNIX)

Algunas llamadas al sistema sobre archivos en UNIX:

```
int creat (file_path, flags); // creacion
int open (file_path, flags); // apertura -> file descriptor
int close (fd); // cierre
int read (fd, buffer, count); // lectura desde el cursor
int write (fd, buffer, count); // escritura desde el cursor
int seek (fd, pos, from); // posicionado del cursor
int unlink(path); // eliminacion
int link (dst, name); // hard link
int symlink(dst, name); // symbolic link
int stat (fd, stat_buffer); // metadatos
```

Y sobre directorios:

```
int mkdir(path, mode);
int rmdir(path);
```

Idea intuitiva

`open` retorna un *file descriptor* que se usa en todas las operaciones siguientes. Posiciona el cursor en cero al abrir en modo lectura/escritura, o al final si se abre en modo `APPEND`. `read` retorna 0 cuando el cursor estaba en EOF.

Para acceder al *contenido* de un directorio, éste puede leerse como un archivo cuyos registros son los metadatos de los archivos que contiene. La biblioteca estándar de cada SO ofrece funciones para manipular directorios (en UNIX, ver `readdir()`).

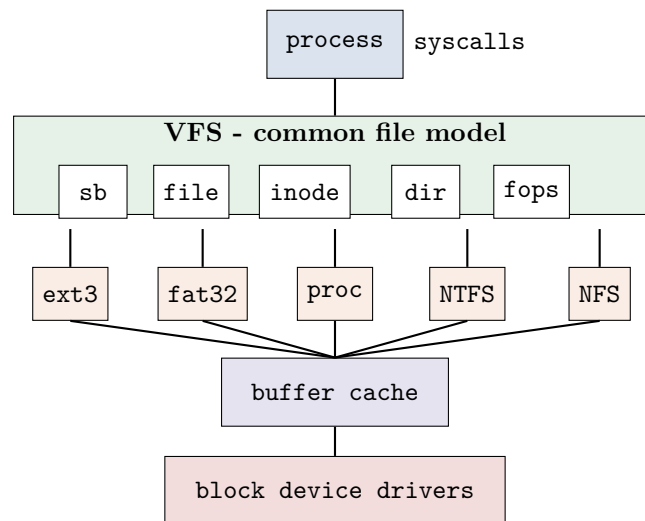
4. Virtual File System (VFS)

Los SO modernos soportan diferentes tipos (o formatos) de sistemas de archivos en distintos dispositivos y tecnologías. En estos casos debe presentarse al usuario un sistema de archivos *uniforme* con operaciones que oculten los detalles de bajo nivel.

Definición

El **Virtual File System (VFS)** es el subsistema del kernel que, ante una llamada al sistema sobre archivos, determina *sobre qué dispositivo y con qué sistema de archivos concreto* debe realizarse la operación. Define un *common file model* basado — en los UNIX — en nodos índices (*inodes*).

4.1. Diseño del VFS



4.2. Estructuras de datos del common file model

El VFS define una interfaz que cada FS concreto debe implementar y estructuras de datos comunes que ocultan los detalles de cada filesystem real. Cada implementación concreta debe **mapear** su representación al modelo abstracto.

- **Superblock (sb).** Metadatos del filesystem: tipo o id, número y tamaño de bloques, ubicación del directorio raíz, etc.
- **File.** Representa un archivo *abierto por un proceso*. Contiene al menos un puntero al *inode* del archivo y un *offset* (cursor de lectura/escritura).
- **Inode.** Contiene los *metadatos* del archivo: tamaño, dispositivo, bloques de datos usados, etc.
- **Directory (dir).** Un directorio es un archivo de datos cuyas entradas son pares (*name*, *inode number*). El VFS define también una interfaz abstracta para buscar por *file paths*.
- **fops.** Interfaz abstracta que cada filesystem concreto debe implementar con operaciones como *open*, *read/write* y *free* sobre inodes.

4.3. Servicios comunes provistos por el VFS

- Punto de entrada de las llamadas al sistema (*open*, *read*, *write*, ...).
- **Buffers de caché** para dispositivos de bloques. Los bloques de datos se mantienen en una caché con *pool* limitado y políticas de reemplazo (LRU, similar a memoria virtual). *No* se usa caché sobre dispositivos de caracteres.
- **Gestión de los requerimientos de E/S**, basada en colas y algoritmos de ordenamiento (algoritmo del elevador para discos, FIFO para ttys).
- Registración de los sistemas de archivos concretos.
- Registración de los device drivers.
- Montaje y desmontaje de filesystems concretos.

Importante

El módulo de *buffers de caché* es el que se comunica con los *block device drivers*. Los filesystems usan sus servicios para leer/escribir bloques: nunca tocan directamente al driver.

5. Representación de archivos y directorios

Para gestionar archivos y directorios se necesitan estructuras de control y metadatos almacenadas en el propio dispositivo. Mínimamente:

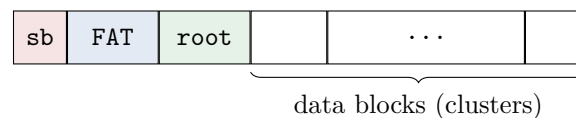
- Metadatos del FS (ubicación del bloque del directorio raíz, número de bloques en el dispositivo, etc.).
- Conjunto de bloques libres y usados.
- Descripción de espacios de bloques de datos y de metadatos.

La forma concreta que toman esas estructuras se llama *formato del sistema de archivos*. Crear un sistema de archivos inicial se conoce como **formatear** el dispositivo (comandos **mkfs** en UNIX o **format** en DOS/Windows).

A continuación se analizan dos formatos ampliamente usados: FAT e inodes.

5.1. FAT

FAT (*File Allocation Table*) fue desarrollado para DOS y se usó en las primeras versiones de MS-Windows. Su estructura:



Superblock. Contiene metadatos como el tipo de FS (*FAT magic number*), el número de bloques del volumen, el tamaño de bloque, etc.

Clusters. Para reducir el número de bloques y la fragmentación de datos, los bloques físicos se agrupan en **clusters**: grupos de bloques contiguos (típicamente de 1, 2, 4, 8 o 16 bloques). Un cluster es la *unidad mínima* de asignación de datos para un archivo. El primer cluster de datos contiene el directorio raíz. En lo que sigue, “bloque” y “cluster” se usan como sinónimos.

La tabla FAT. `fat[N]` es un arreglo con tantas entradas como clusters de datos. Implementa *listas enlazadas* de los bloques de datos de cada archivo. Un directorio se representa como un archivo cuya secuencia de registros tiene la forma:

file name	attrs	cluster1
-----------	-------	----------

En FAT32 cada entrada tiene 32 bits con 12 bytes para el nombre y bits de atributos (*size, read only, hidden, fechas, ...*).

Lectura de un archivo. `cluster1` contiene el número del primer cluster del archivo. Los siguientes clusters se obtienen indexando en la FAT: `FAT[cluster1]` contiene el *next cluster*. Un valor 0 indica fin del archivo. Así, un archivo con tres clusters $j \rightarrow i \rightarrow ?$ se representa con $FAT[j] = i$, $FAT[i] = 0$ (terminador).

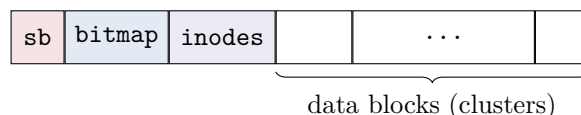
Idea intuitiva

La FAT contiene la lista de clusters de cada archivo. Los clusters *libres* pueden inferirse de la FAT: si un cluster no es alcanzable desde ningún directorio, está libre. No hace falta almacenarlos en disco; pueden calcularse al inicio y mantenerse en memoria.

5.2. Nodos índices (inodes)

Una técnica alternativa, usada por la mayoría de los FS para grandes volúmenes (**ext2/3/4** en GNU/Linux y otros UNIX), *separa* los metadatos de la entrada del directorio. Un **inode** (*index node*) contiene los metadatos del archivo más la lista de bloques de datos que usa.

Estructura del filesystem.



Contenido de un inode. Un inode contiene:

- Atributos (dueño, permisos, tamaño en bytes, fechas, ...).
- Un arreglo o tabla de números de bloques de datos (*data blocks table*) de tamaño *predefinido* k , que representa la secuencia de bloques que ocupa el archivo.

Directorios. Un directorio es un archivo estructurado cuyas entradas tienen sólo dos campos: (*filename*, *inode*). Habitualmente el directorio raíz está representado por el primer inode. Los inodes están numerados/indexados.

Gestión de espacios libres. Es conveniente usar un **bitmap** para la gestión de inodes y bloques de datos libres. Si bien se podrían deducir por *complemento* de los alcanzables, esto sería costoso porque la información está distribuida en todos los inodes.

Tamaño de un inode. Por eficiencia se elige el tamaño de un inode igual al de un bloque físico (sector) del disco.

Importante

Un inode con tabla de tamaño k puede representar archivos de *a lo sumo* k bloques de datos. Para archivos más grandes se usa la técnica de los **inodes indirectos**.

Inodes indirectos. La idea es que las últimas entradas de la tabla del inode apunten a *otros inodes* que sólo contienen tablas de números de bloques adicionales. Comúnmente las entradas finales son:

- **Indirecto simple:** árbol de un nivel.
- **Indirecto doble:** árbol de dos niveles.
- **Indirecto triple:** árbol de tres niveles.

Cuantos más niveles de indirección, más grande puede ser el archivo, pero más lentos son los accesos a los últimos bloques (más indirecciones a resolver).

En la creación del filesystem. Al ejecutar **mkfs** se decide la cantidad de inodes a reservar en el disco. El número de inodes determina la *cantidad máxima de archivos y directorios* que puede tener el filesystem.

6. Montaje de sistemas de archivos

Los SO presentan el espacio de nombres global de dos maneras distintas:

- **MS-Windows.** Cada FS aparece de forma *independiente*: un full path toma la forma de un nombre de dispositivo (una letra) seguido por dos puntos y un path. Típicamente **C** es el disco primario.
- **UNIX.** Se presenta al usuario un **único árbol** de archivos. La raíz corresponde al FS del dispositivo de almacenamiento primario. Para acceder a otro dispositivo hay que **montarlo** en algún directorio (por ejemplo `/mnt/mypendrive`) del árbol. Los comandos son `mount` y `umount`.

Idea intuitiva

El mecanismo de montaje permite que las aplicaciones accedan a archivos con nombres pre-establecidos sin nombrar dispositivos (**C:**, **D:**). Esto facilita escribir aplicaciones portables.

6.1. Ejemplo: montar un pendrive FAT en Linux

```
mkdir /mnt/mypendrive
mount /dev/sdb /mnt/mypendrive
```

Esta operación “cuelga” el FS del pendrive en el directorio `/mnt/mypendrive` del FS raíz. Luego, los archivos del pendrive son accesibles mediante el path `/mnt/mypendrive/....`

6.2. Tabla de montaje

El VFS mantiene una tabla de filesystems montados, por ejemplo:

mount point	fs type	device	major	access mode
/	ext4	/dev/sda1	1	rw
/mnt/mypendrive	fat32	/dev/sdb	5	r
/proc	procfs	proc	0	rw
...	...	...	...	...

En UNIX, esta información se obtiene con el comando `mount` o leyendo `/etc/mtab`.

umount. El comando `umount /mnt/mypendrive` desmonta el filesystem. Es seguro retirar el dispositivo recién *después* de desmontarlo porque hace un **flush** de los bloques que aún pudieran estar en caché.

6.3. Interfaz de FS que el VFS exige

Cada FS concreto debe implementar y registrar:

- **mount:** cargar y registrar el superblock.
- **umount:** liberar el superblock.
- Operaciones sobre directorios (creación, destrucción, búsqueda por file paths, ...).
- Operaciones sobre archivos (inodes).

Cada proceso registra sus archivos abiertos en una tabla de su propio descriptor (en xv6 es el campo `ofile[]` de `struct proc`), representada como un arreglo de punteros a entradas en la tabla de `struct file`.

6.4. Anatomía de un open

Cuando un proceso ejecuta `fd = open("/home/user1/docs/mycv.tex", ...)`, el VFS:

1. Determina en qué FS concreto está el archivo (en este caso el FS raíz, de tipo `ext4`).

2. Invoca una función del tipo `inode = fs.dir_lookup("/home/user1/docs/mycv.tex")` que busca y lee el inode correspondiente.
3. Crea una nueva entrada en la tabla de archivos abiertos del sistema.
4. Registra el archivo abierto en el descriptor del proceso.
5. Retorna el índice en `ofiles[]` (el *file descriptor*).

6.5. Anatomía de un write

Cuando luego el proceso ejecuta `write(fd, buffer, count)`, el VFS:

1. Accede a la `struct file` correspondiente a `fd`.
2. De la tabla de montaje determina el FS concreto a usar (por ejemplo, el módulo `ext4`).
3. Invoca una función del tipo `fs.write(inode, buffer, count, offset)`. Esta requerirá escribir en uno o más bloques, servicio que provee la capa de *buffer de caché* mediante `bio_write(buf, dev)`.
4. Si `buf` no está en la caché, hay que cargarlo del disco en un nuevo buffer; luego se modifica y se marca como *dirty* (se escribirá a disco en un `close(fd)` o cuando expire un timer).
5. Si hay que hacer un requerimiento a disco, éste se encola según una política de scheduling (algoritmo del elevador).
6. El *I/O scheduler* eventualmente invoca a `block_dev.do_request(request)` del device driver registrado. El *major number* del inode determina cuál es el driver; el *minor number* le dice al driver con qué dispositivo concreto interactuar.

Idea intuitiva

Como un device driver puede controlar varios dispositivos del mismo tipo, la pareja (major, minor) es la clave: major elige el driver y minor elige el dispositivo concreto dentro de ese driver.

7. Dispositivos como archivos especiales

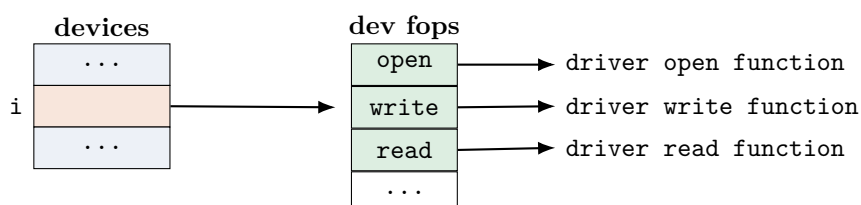
En muchos SO los dispositivos aparecen en el sistema de archivos como **archivos especiales**. En UNIX viven bajo `/dev` (por *devices*). Estos archivos le permiten al VFS mantener la relación entre los dispositivos y los device drivers correspondientes.

Major y minor numbers. Cada archivo especial contiene un par de campos numéricos:

- **major number:** determina el *tipo* de dispositivo. Se usa para relacionar una clase de dispositivos con su device driver.
- **minor number:** identifica un dispositivo dentro de ese tipo.

7.1. Tabla de device drivers registrados

Cada device driver se registra en el VFS en su inicio, indicando el archivo especial y el major number. Así, el VFS mantiene una tabla:



Cada entrada se indexa por el *major number*. Asocia el dispositivo con las operaciones registradas por el driver.

8. Descriptor de archivo

`open` retorna un **file descriptor** que queda registrado en la *opened files table* (`ofiles`) del descriptor del proceso. Esta tabla es un arreglo de punteros a estructuras como:

```
struct file {
    union {
        struct pipe *pipe;
        struct inode *inode;
    };
    int ref_count; // procesos trabajando con este file
    unsigned int offset; // posicion actual de lectura/escritura
};
```

Un inode puede representarse como:

```
struct inode {
    unsigned int num; // numero de inode (en disco)
    int ref_count;
    // los campos siguientes se traen/guardan de/a disco
    short int type; // file, directory, FIFO, device, ...
    short int major;
    short int minor;
    short int links; // numero de symlinks a este inode
    // otros metadatos: owner id, group id, permisos, ...
    unsigned int size; // en bytes
    unsigned int data_blocks[N];
};
```

9. Operaciones con dispositivos de caracteres

La E/S con dispositivos de caracteres se diferencia de la de bloques en dos aspectos:

1. No tienen filesystem asociado.
2. La E/S **no pasa por la caché de bloques** (no es buffered): se realiza *directamente* contra el dispositivo.

9.1. Ejemplo: escribir a stdout

Cuando un proceso desea escribir en el descriptor 1 (salida estándar), ese descriptor refiere a una `struct file` cuyo `inode` describe un *archivo especial* (por ejemplo `/dev/tty` en Linux o `console` en xv6) representando un dispositivo de caracteres. El *major number* identifica al driver con el que se debe interactuar. El VFS invoca a la función correspondiente:

```
devices[p->ofiles[1]->inode.major].write(p->ofiles[1], ...);
```

9.2. Caso xv6

En xv6 (similar a UNIX) los character device drivers se registran en `struct devsw devsw[]`. En xv6-riscv hay un único dispositivo registrado: la **consola**. La comunicación se hace por el puerto serie UART:

- `uart.c` implementa el *bottom half* (bajo nivel).
- `console.c` implementa la lógica de alto nivel (*tty discipline*) del driver.

El archivo `console` en la raíz del FS es un archivo especial que representa al dispositivo. Es abierto por `init` **tres veces** para entrada estándar, salida estándar y salida estándar de errores. Esos descriptores son *heredados* por los demás procesos (shell y otros) vía `fork()`.

10. Filesystem de xv6

Xv6 monta un *único* sistema de archivos con su propio formato basado en inodes. Se construye al compilar los procesos de usuario mediante la utilidad `mkfs` incluida en el build-system.

10.1. Implementación por capas

File descriptor	file.h/c
Pathname	
Directory	fs.h/c
Inode	
Logging	log.c
Buffer cache	bio.c
Disk driver	virtio disc.c

10.2. FS transaccional con logging

Importante

El filesystem de xv6 es **transaccional** para evitar inconsistencias en *crashes* o cortes de energía. Antes de escribir un conjunto de requerimientos a disco, xv6 los escribe en un **log** (área especial). Luego realiza un *commit*, replicando (o moviendo) las operaciones del log a los bloques correspondientes. Finalmente borra el log.

Recuperación tras crash. Cuando el sistema inicia, el FS verifica el log:

- Si está marcado como *completado*, se realizan las copias a los bloques de datos contenidos en el log.
- Si no, el log se ignora y se elimina. En este caso el sistema pierde las últimas escrituras, pero queda en un estado **consistente**.

10.3. Buffer cache simplificada

El buffer de caché de xv6 consiste en una **lista doblemente encadenada** de bloques (buffers). Los buffers son fijos (arreglo declarado estáticamente) y *no* implementa una técnica de reemplazo. El manual de xv6 describe en mayor detalle el diseño y la implementación.

11. Booting

El proceso de inicio (*boot*) comúnmente comienza por un programa en ROM conocido como **firmware**.

11.1. BIOS

En la PC el firmware se denomina **Basic Input-Output System (BIOS)**. Consiste en un conjunto de programas que verifica la RAM física instalada y algunos dispositivos básicos (teclado, discos). Suele almacenarse en ROM o EPROM. Usa una memoria volátil mantenida con una pequeña batería, conocida como **CMOS**, donde guarda fecha, hora y parámetros de configuración del hardware.

Carga del kernel. La BIOS carga en memoria el *primer sector* del disco primario (el **boot sector**) y le transfiere el control. Este programa **boot loader** se encarga de cargar en memoria el código y los datos del kernel del SO y transferirle el control.

El boot loader es muy pequeño (debe caber en un bloque de típicamente 512 bytes); ese bloque también puede contener la tabla de particiones. El programa debe saber dónde se encuentra el kernel en el disco o volumen.

11.2. Multiboot loaders (GRUB)

Un **boot loader genérico multiboot** como **GRUB** permite iniciar diferentes sistemas operativos instalados en distintos dispositivos o volúmenes. Realiza la tarea en *dos etapas*:

1. El boot loader del boot sector carga el multiboot loader (GRUB) y le transfiere el control.
2. GRUB — que conoce formatos de filesystems y de ejecutables (ELF, COFF, etc.) — lee un archivo de configuración y presenta un menú al usuario para que seleccione el SO a iniciar.

Estos multiboot loaders se usan para iniciar SO tipo UNIX (como GNU/Linux), donde el kernel es un archivo más cuya ubicación dentro del FS no se conoce a priori.

El método descrito se conoce como **BIOS booting** y se desarrolló en 1975, principalmente para la PC y DOS.

11.3. UEFI

Actualmente es común el estándar **Unified Extensible Firmware Interface (UEFI)**, basado en un mecanismo mucho más flexible. Sus características:

- Para que sea booteable, un disco debe contener una partición de tipo FAT rotulada *EFI*. En ella se almacenan directamente los kernels de los SO o arrancadores (como GRUB) y sus archivos de configuración. UEFI conoce el formato FAT y carga el SO en base a esos archivos.
- Define en qué áreas de memoria el kernel encontrará información sobre la configuración del hardware.
- Cada kernel (o boot loader) es un archivo con extensión *.efi* compilado y enlazado para soportar el estándar.
- **Secure boot**: soporta firmas de kernels para verificar su integridad. El firmware permite almacenar en la EPROM certificados con las claves públicas de las empresas u organizaciones que los desarrollan.
- Soporta booteo remoto y tablas de partición de tamaño arbitrario basadas en el estándar **GPT** (*Guided Partition Table*).

12. Sistemas de archivos especiales

Algunos SO modernos como GNU/Linux soportan sistemas de archivos cuyo contenido *no necesariamente se almacena en disco*. Se conocen como **pseudo filesystems** o *especiales*.

12.1. procfs

En GNU/Linux se usa **procfs**, comúnmente montado en **/proc**. Brinda información sobre datos del kernel y de cada proceso en ejecución, en forma de un árbol de directorios y archivos. Los usuarios pueden acceder y modificar datos (variables) internos del kernel como si fueran archivos:

```
cat /proc/cpuinfo # detalles de las CPUs
cat /proc/meminfo # uso de la memoria
```

Salida típica de `cat /proc/meminfo`:

```
MemTotal: 994548 kB
MemFree: 65228 kB
MemAvailable: 263724 kB
Buffers: 21396 kB
Cached: 304440 kB
SwapCached: 25260 kB
Active: 267424 kB
Inactive: 503720 kB
Active(anon): 110956 kB
Inactive(anon): 351176 kB
...
```

Es posible modificar parámetros del kernel mediante:

```
echo <value> > /proc/sys/<class>/<parameter>
```

12.2. sysfs

sysfs está diseñado específicamente para contener archivos especiales que representan dispositivos. Comúnmente se monta en **/dev**. Simplifica la manipulación de parámetros del kernel y de dispositivos/drivers. En sistemas tipo BSD-UNIX o Linux se logra lo mismo con el comando (y syscall) `sysctl`.

12.3. ramfs e initramfs

Otros filesystems se implementan en memoria principal, como **ramfs**. Se usan durante el *booting* como sistema de archivos raíz inicial: el boot loader carga en memoria una imagen del filesystem y se la presenta al kernel como un **initramfs**. Estos filesystems contienen un conjunto reducido de programas de inicio y de montaje de otros sistemas de archivos en disco.

busybox. En Linux, el *initramfs* contiene comúnmente un FS con las utilidades del paquete **busybox**, que contiene versiones mínimas de las utilidades más comunes de UNIX en un único ejecutable. Busybox también se usa en *embedded systems*, ya que permite generar un FS muy pequeño con algunos archivos de configuración en **/etc**, dispositivos en **/dev/** y un kernel (generalmente Linux).

13. Cierre

El subsistema de archivos es la capa que organiza la persistencia y le da forma humana al almacenamiento masivo. Las ideas más importantes:

- Un disco se ve a través de **volúmenes**; cada uno contiene un *filesystem* con su propio formato (FAT, ext, NTFS, ...).

- Cada archivo es un par (*attributes*, *data*) y se organiza dentro de un **árbol de directorios**. La unidad de identificación es el *path*.
- El **VFS** provee un *common file model* basado en **sb**, **file**, **inode**, **dir** y **fops**. Cada FS concreto mapea su formato a este modelo y se enchufa al kernel implementando una API definida.
- Dos formatos paradigmáticos: **FAT** (listas enlazadas de clusters indexadas en la tabla FAT) e **inodes** (cada archivo es un nodo índice con metadatos + tabla de bloques; para archivos grandes se usan inodes indirectos simple/doble/triple).
- El **montaje** cuelga otro FS en algún directorio del árbol raíz. La tabla de montaje del VFS le permite enrutar cada operación al filesystem correcto.
- Bajo el principio Unix de “todo es un archivo”, los dispositivos viven como **archivos especiales** en `/dev`, con un par (major, minor): major elige el driver, minor elige el dispositivo concreto. El descriptor de archivo del proceso (**ofiles**) apunta a una **struct file** con cursor compartido.
- Los **dispositivos de caracteres** no usan caché de bloques: la E/S va directa al driver. Ejemplo paradigmático: la consola de xv6, abierta tres veces por **init** y heredada vía **fork**.
- El **filesystem de xv6** está organizado en capas (file descriptor / pathname / directory / inode / logging / buffer cache / disk driver). Es *transaccional*: el log garantiza que un crash deje el FS en estado consistente.
- El **booting** arranca con el firmware (BIOS) que carga un boot loader (eventualmente multi-boot, como GRUB); el estándar moderno UEFI usa una partición FAT con archivos **.efi** y soporta *secure boot* y GPT.
- Los **filesystems especiales** (procfs, sysfs, ramfs, initramfs con busybox) exponen información o servicios del kernel como si fueran archivos: el mismo principio uniforme que con los dispositivos.

Idea intuitiva

La gran abstracción del sistema de archivos es presentar a procesos y usuarios un *único árbol uniforme* en el que conviven discos, particiones, dispositivos, parámetros del kernel y procesos en ejecución. Esa uniformidad es la base que permite construir, encima, un shell expresivo y herramientas Unix simples y componibles.